

École Supérieure de Technologie - Meknès

Département Informatique

Filière : IATE/S3

Module : UML & Java Avancé

Système de Gestion de Bibliothèque Universitaire

Informatisation Complète de la Bibliothèque

Réalisé par :

Abdelhakim LAACHACHI
Abdenmour TIJENTE
Mohamed FETTACHE
Ilyass NAJIM

Encadré par :

Pr. Chaymaa LAMINI

Année Universitaire 2025-2026

Remerciements

Nous tenons à exprimer notre profonde gratitude à notre encadrante, **Professeur Chaymaa LAMINI**, pour son soutien constant, ses conseils précieux et son expertise qui ont grandement contribué à la réalisation de ce projet. Enfin, nous remercions tous ceux qui, de près ou de loin, ont contribué à la réussite de ce projet.

Résumé

Ce rapport présente la conception et l'implémentation d'un système informatisé de gestion de bibliothèque universitaire pour l'École Supérieure de Technologie de Meknès. Actuellement, la bibliothèque gère ses processus de manière entièrement manuelle, ce qui entraîne des pertes de données, des erreurs de suivi et une absence de traçabilité. Face à ces limitations, ce projet propose une solution complète permettant d'automatiser et d'optimiser l'ensemble des processus bibliothécaires. Le système développé en Java utilise une architecture hybride innovante : **JavaFX** pour l'interface administrateur moderne et **Swing** pour l'interface adhérent classique. Il couvre la gestion intégrale des livres, des adhérents et des opérations d'emprunt/retour, avec un système d'authentification sécurisé basé sur BCrypt et une traçabilité complète via logs. L'application suit une architecture en couches (Model-DAO-Service-Controller-UI) garantissant maintenabilité et évolutivité. Elle implémente toutes les règles de gestion métier : limitation des emprunts, calcul automatique des dates de retour, détection des retards, et blocage automatique des adhérents défaillants. Le projet a été développé en suivant les méthodologies agiles, avec gestion de version Git/GitHub et tests complets. L'application est packagée sous forme de JAR exécutable et déployable facilement. **Mots-clés** : Gestion de bibliothèque, Java, JavaFX, Swing, UML, MySQL, Architecture en couches, Maven, Git.

Table des matières

Remerciements	i
Résumé	ii
Introduction	1
1 Analyse et Conception UML	3
1.1 Introduction	3
1.2 Identification des Acteurs	3
1.2.1 Administrateur	3
1.2.2 Utilisateur (Adhérent)	3
1.3 Diagramme de Cas d'Utilisation	4
1.3.1 Vue d'ensemble	4
1.3.2 Cas d'utilisation de l'Administrateur	4
1.3.3 Cas d'utilisation de l'Utilisateur	5
1.3.4 Relations entre cas d'utilisation	5
1.4 Diagramme de Classes	6
1.4.1 Présentation	6
1.4.2 Classes principales	6
1.4.3 Relations entre classes	7
1.4.4 Contraintes métier	8
1.5 Diagrammes de Séquence	8
1.5.1 Séquence : Authentification	8
1.5.2 Séquence : Enregistrer un emprunt	9
1.5.3 Séquence : Retourner un livre	9
1.6 Synthèse de la conception	10
2 Architecture et Structure du Projet	11
2.1 Introduction	11
2.2 Architecture en Couches	11
2.2.1 Vue d'ensemble	11
2.2.2 Description détaillée des couches	12
2.3 Structure du Projet (Arborescence)	14
2.4 Gestion de la Base de Données	15
2.5 Synthèse	15
3 Implémentation et Technologies Utilisées	16
3.1 Introduction	16
3.2 Technologies et Outils	16
3.3 Points Clés d'Implémentation	16

3.3.1	Authentification Sécurisée	16
3.3.2	Interface Administrateur	16
3.3.3	Interface Adhérent	17
3.3.4	Logging Complet	17
3.4	Défis Rencontrés et Solutions	17
3.5	Synthèse	17
4	Résultats et Preuves de Réalisation	18
4.1	Interfaces Graphiques	18
4.1.1	Interface Administrateur	18
4.1.2	Interface Adhérent	19
4.2	Tests et Validation	20
4.3	Packaging et Déploiement	20
5	Conclusion et Perspectives	21
5.1	Conclusion	21
5.2	Perspectives d'Amélioration	21

Table des figures

1.1	Diagramme de cas d'utilisation global du système	4
1.2	Diagramme de classes du système	6
1.3	Diagramme de séquence - Authentification	8
1.4	Diagramme de séquence - Emprunt d'un livre	9
2.1	Architecture en couches du système	12
4.1	Dashboard principal de l'administrateur (onglets Livres, Adhérents, Emprunts)	18
4.2	Gestion du catalogue (modification)	19
4.3	Vue du catalogue pour un adhérent (mode consultation uniquement)	19
4.4	Confirmation d'une emprunt	20

Liste des tableaux

1.1	Relations principales entre les classes	7
2.1	Schéma principal de la base de données	15
3.1	Stack technologique du projet	16

Introduction

Contexte général

L'École Supérieure de Technologie de Meknès dispose d'une bibliothèque universitaire qui constitue une ressource essentielle pour les étudiants et le corps enseignant. Actuellement, l'ensemble des opérations de gestion s'effectue de manière manuelle : inscription des adhérents, gestion du catalogue de livres, enregistrement des emprunts et retours, suivi des retards, et inventaire du stock. Cette gestion manuelle présente de nombreuses limitations :

- **Pertes de données** : Risques de perte ou détérioration des fiches papier
- **Erreurs de suivi** : Difficultés à tracer les emprunts et identifier les retards
- **Inefficacité** : Temps de traitement élevé pour les opérations courantes
- **Absence de statistiques** : Impossibilité de générer des rapports d'activité
- **Manque de traçabilité** : Aucun historique des opérations effectuées

Problématique

Face à l'augmentation constante du nombre d'étudiants et de la collection de livres, la gestion manuelle atteint ses limites. Il devient urgent de mettre en place un système informatisé permettant :

- De centraliser toutes les données dans une base unique
- D'automatiser les processus répétitifs
- De garantir la traçabilité complète des opérations
- D'améliorer l'efficacité du personnel
- D'offrir un meilleur service aux utilisateurs

Objectifs du projet

Ce projet vise à développer une application Java Desktop complète permettant de :

1. **Gérer le catalogue** : Ajout, modification, suppression et recherche de livres
2. **Gérer les adhérents** : Inscription, consultation et gestion des blocages
3. **Gérer les emprunts** : Enregistrement, retours, calcul des retards et amendes
4. **Assurer la sécurité** : Authentification robuste et contrôle d'accès par rôles
5. **Garantir la traçabilité** : Système de logs complet pour toutes les opérations
6. **Faciliter l'utilisation** : Interfaces graphiques modernes et intuitives

Méthodologie adoptée

Le développement suit une approche structurée en plusieurs phases :

- **Analyse** : Identification des besoins et spécification des exigences
- **Conception** : Modélisation UML complète (cas d'utilisation, classes, séquence)
- **Implémentation** : Développement en Java avec architecture en couches
- **Tests** : Validation fonctionnelle et tests d'intégration
- **Déploiement** : Packaging et documentation

Organisation du rapport

Ce rapport s'organise en cinq chapitres principaux :

Chapitre 1 Analyse et Conception UML du système

Chapitre 2 Architecture et Structure du Projet

Chapitre 3 Implémentation et Technologies

Chapitre 4 Résultats et Preuves de Réalisation

Chapitre 5 Conclusion et Perspectives

Chapitre 1

Analyse et Conception UML

1.1 Introduction

La modélisation UML (Unified Modeling Language) constitue une étape fondamentale dans le processus de développement. Elle permet de représenter visuellement l'architecture du système avant son implémentation, facilitant ainsi la communication entre les membres de l'équipe et servant de référence pour le développement.

1.2 Identification des Acteurs

Le système de gestion de bibliothèque implique deux catégories principales d'utilisateurs, chacune ayant des rôles et permissions spécifiques.

1.2.1 Administrateur

Rôle : Gestionnaire complet du système **Responsabilités :**

- Gestion complète du catalogue de livres (CRUD)
- Gestion des adhérents (inscription, modification, blocage)
- Enregistrement des emprunts et retours
- Gestion des comptes utilisateurs
- Consultation des logs et génération de rapports
- Configuration des paramètres système

Interface : Dashboard JavaFX moderne avec navigation intuitive

1.2.2 Utilisateur (Adhérent)

Rôle : Membre de la bibliothèque avec accès limité **Responsabilités :**

- Consultation du catalogue de livres
- Recherche de livres par différents critères
- Consultation de ses propres emprunts
- Demande de réservation de livres
- Consultation de son historique

Interface : Application Swing classique et ergonomique

1.3 Diagramme de Cas d'Utilisation

1.3.1 Vue d'ensemble

Le diagramme de cas d'utilisation offre une représentation graphique des fonctionnalités du système.

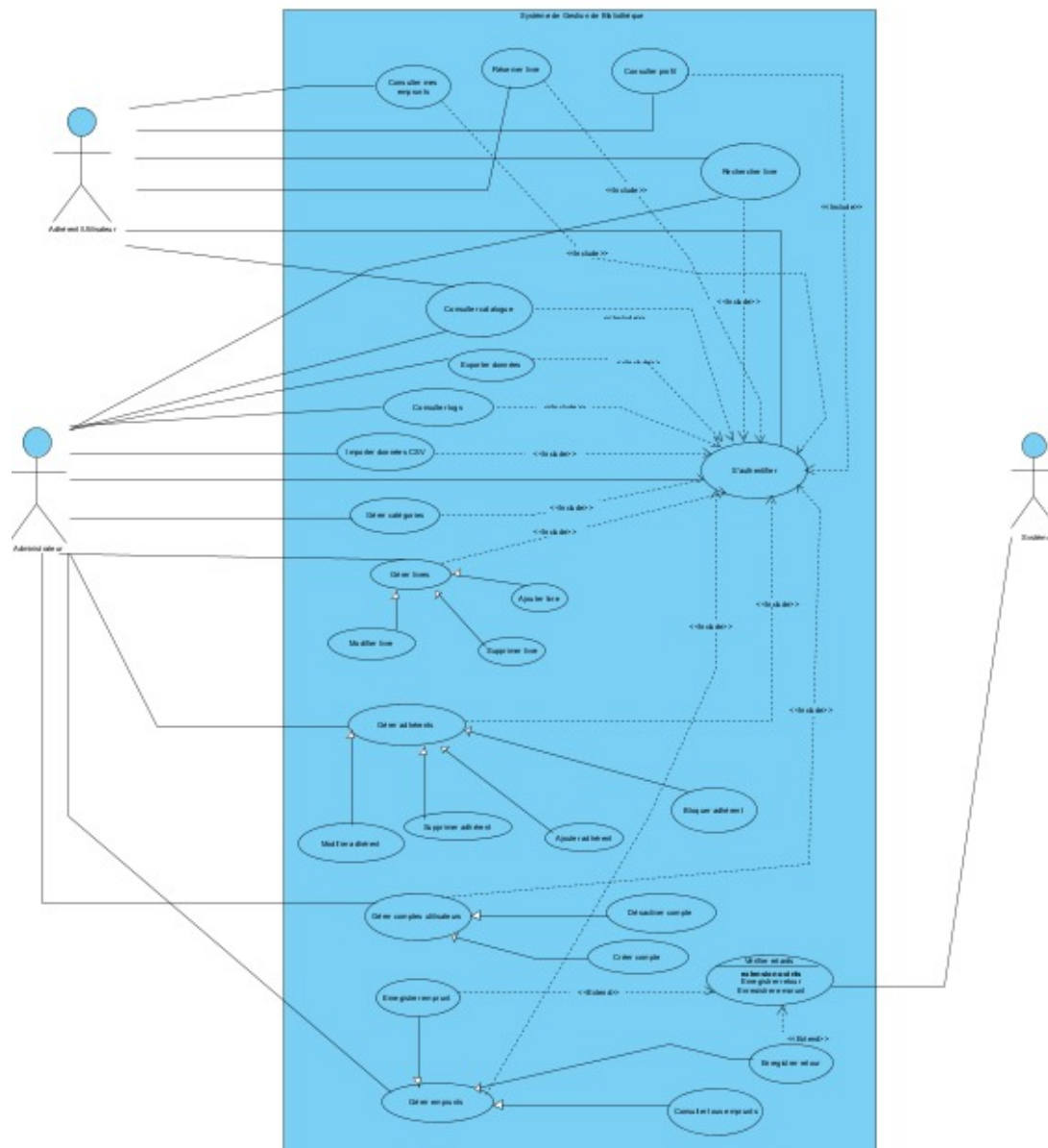


FIGURE 1.1 – Diagramme de cas d'utilisation global du système

1.3.2 Cas d'utilisation de l'Administrateur

Module Gestion des Livres

- **Ajouter un livre** : Créer une nouvelle entrée dans le catalogue
- **Modifier un livre** : Mettre à jour les informations existantes

- **Supprimer un livre** : Retirer un livre (si aucun emprunt actif)
- **Rechercher un livre** : Recherche multicritère (titre, auteur, ISBN, catégorie)
- **Gérer les catégories** : Organiser le catalogue par thématiques

Module Gestion des Adhérents

- **Inscrire un adhérent** : Créer un nouveau compte membre
- **Modifier un adhérent** : Mettre à jour les informations personnelles
- **Consulter un adhérent** : Voir le profil et l'historique
- **Bloquer/Débloquer** : Gérer l'accès aux emprunts

Module Gestion des Emprunts

- **Enregistrer un emprunt** : Créer un nouvel emprunt avec vérifications
- **Enregistrer un retour** : Traiter le retour avec calcul de retard éventuel
- **Consulter les emprunts** : Visualiser tous les emprunts (actifs, retards)
- **Prolonger un emprunt** : Étendre la durée si conditions remplies

1.3.3 Cas d'utilisation de l'Utilisateur

- **S'authentifier** : Connexion avec identifiants personnels
- **Consulter le catalogue** : Parcourir la liste des livres disponibles
- **Rechercher des livres** : Utiliser les critères de recherche
- **Voir détails d'un livre** : Informations complètes et disponibilité
- **Consulter mes emprunts** : Voir ses emprunts en cours et dates de retour
- **Consulter mon historique** : Accéder à l'historique des emprunts passés

1.3.4 Relations entre cas d'utilisation

Relations d'inclusion («include»)

- Tous les cas d'utilisation «**include**» "S'authentifier"
- "Enregistrer un emprunt" «**include**» "Vérifier disponibilité livre"
- "Enregistrer un emprunt" «**include**» "Vérifier statut adhérent"

Relations d'extension («extend»)

- "Enregistrer un retour" «**extend**» "Calculer amende" (si retard)
- "Rechercher un livre" «**extend**» "Réserver un livre" (si non disponible)

1.4 Diagramme de Classes

1.4.1 Présentation

Le diagramme de classes représente la structure statique du système avec les entités, leurs attributs, méthodes et relations.

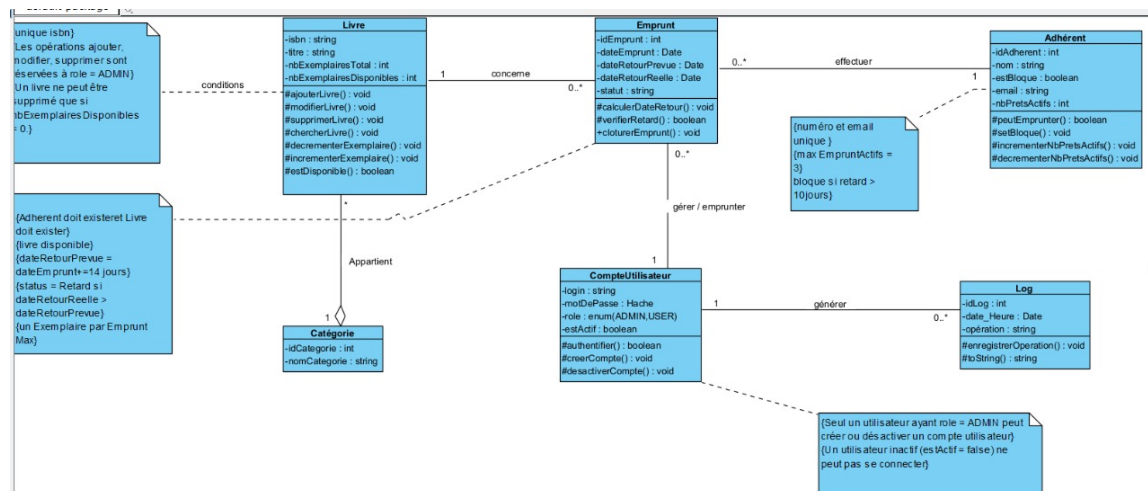


FIGURE 1.2 – Diagramme de classes du système

1.4.2 Classes principales

Classe Livre

Attributs :

- -isbn : String (clé primaire, unique)
- -titre : String
- -auteur : String
- -nombreExemplaires : int
- -nombreDisponibles : int

Méthodes :

- +estDisponible() : boolean
- +emprunter() : void
- +retourner() : void
- +supprimer() : void
-

Classe Adherent

Attributs :

- -id : int (clé primaire, auto-généré)
- -nom : String
- -prenom : String

- -email : String (unique)
- -dateInscription : Date
- -estBloque : boolean

Méthodes :

- +peutEmprunter() : boolean
- +getNombreEmpruntsActifs() : int
- +bloquer() : void
- +debloquer() : void

Classe Emprunt**Attributs :**

- -id : int
- -dateEmprunt : Date
- -dateRetourPrevue : Date
- -dateRetourEffective : Date
- -statut : String

Méthodes :

- +calculerDateRetour() : Date
- +estEnRetard() : boolean
- +calculerAmende() : double

Classe Compte (Utilisateur)**Attributs :**

- -id : int
- -login : String (unique)
- -motDePasseHash : String
- -role : Role (ADMIN, USER)
- -actif : boolean

Méthodes :

- +authentifier(password : String) : boolean
- +estAdmin() : boolean

1.4.3 Relations entre classes

Relation	Description	Multiplicité
Livre - Categorie	Un livre appartient à une catégorie	1-.*
Emprunt - Livre	Un emprunt concerne un livre	1-0..*
Emprunt - Adherent	Un emprunt est fait par un adhérent	0..*-1
Compte - Adherent	Un compte peut être lié à un adhérent	0..1 - 0..1

TABLE 1.1 – Relations principales entre les classes

1.4.4 Contraintes métier

- Un adhérent ne peut avoir plus de 3 emprunts actifs simultanément
- La durée d'emprunt standard est de 14 jours
- Un adhérent avec retard > 10 jours est automatiquement bloqué
- Un livre ne peut être supprimé que si aucun exemplaire n'est emprunté
- Les mots de passe sont stockés sous forme hachée (BCrypt)

1.5 Diagrammes de Séquence

1.5.1 Séquence : Authentification

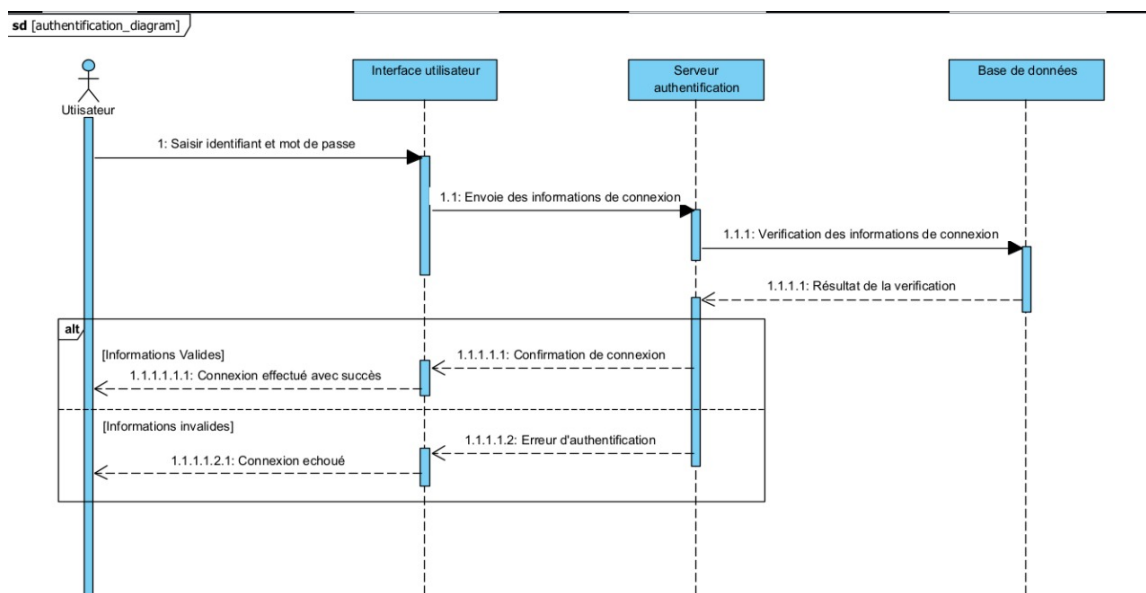


FIGURE 1.3 – Diagramme de séquence - Authentification

Flux :

1. L'utilisateur saisit login et mot de passe
2. Le LoginController appelle LoginService.authentifier()
3. Le LoginService interroge CompteDAO pour récupérer le compte
4. Vérification du hash du mot de passe avec BCrypt
5. Si authentification réussie : création de session et redirection
6. Si échec : affichage d'un message d'erreur

1.5.2 Séquence : Enregistrer un emprunt

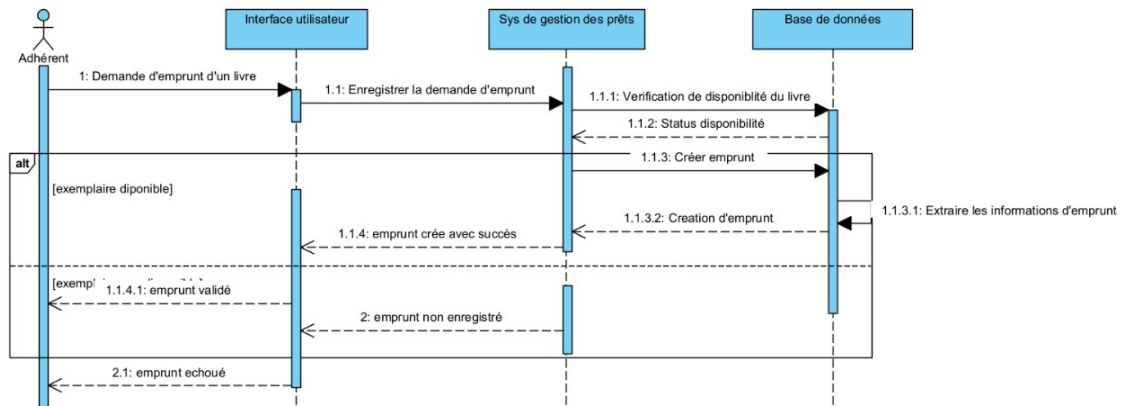


FIGURE 1.4 – Diagramme de séquence - Emprunt d'un livre

Flux :

1. L'admin sélectionne adhérent et livre
2. Le EmpruntController appelle EmpruntService.emprunter()
3. Vérifications : adhérent non bloqué, limite < 3, livre disponible
4. Création de l'emprunt avec calcul de date de retour (+14 jours)
5. Décrémenter des exemplaires disponibles
6. Enregistrement dans la base de données
7. Enregistrement dans les logs
8. Affichage de confirmation

1.5.3 Séquence : Retourner un livre

Flux :

1. L'admin sélectionne un emprunt et clique sur "Retour"
2. Le EmpruntController appelle EmpruntService.retourner()
3. Calcul du nombre de jours de retard
4. Si retard > 0 : calcul de l'amende selon le barème
5. Si retard > 10 jours : blocage automatique de l'adhérent
6. Incrémenter des exemplaires disponibles
7. Mise à jour de l'emprunt (statut, date retour effective)
8. Enregistrement dans les logs
9. Affichage du récapitulatif

1.6 Synthèse de la conception

La modélisation UML nous a permis de :

- Identifier clairement les acteurs et leurs rôles
- Définir toutes les fonctionnalités du système
- Structurer les données avec un diagramme de classes cohérent
- Spécifier les interactions entre objets via les diagrammes de séquence
- Valider les contraintes métier et règles de gestion

Cette conception solide a servi de base pour l'implémentation décrite dans les chapitres suivants.

Chapitre 2

Architecture et Structure du Projet

2.1 Introduction

L'architecture logicielle définit l'organisation du code et la manière dont les différents composants interagissent. Pour ce projet, nous avons adopté une architecture en couches qui favorise la séparation des responsabilités, la maintenabilité et l'évolutivité.

2.2 Architecture en Couches

2.2.1 Vue d'ensemble

Notre application suit une architecture en 6 couches distinctes :

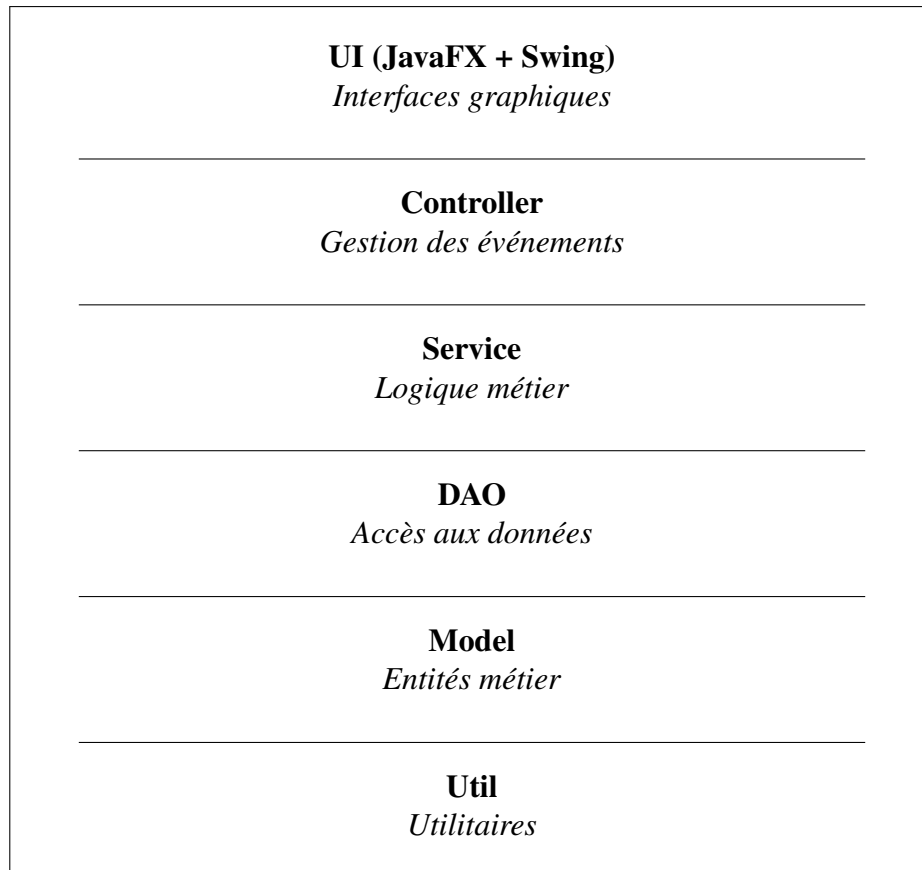


FIGURE 2.1 – Architecture en couches du système

2.2.2 Description détaillée des couches

Couche Model (Entités)

Responsabilité : Définir les entités de données (POJO) correspondant aux tables MySQL
Classes principales :

- Livre.java : Représente un livre du catalogue
- Adherent.java : Représente un membre de la bibliothèque
- Emprunt.java : Représente une transaction d'emprunt
- Categorie.java : Représente une catégorie de livres
- utilisateur.java : Représente un compte utilisateur
- Log.java : Représente une entrée de log

Caractéristiques :

- Classes POJO avec attributs privés
- Getters et setters pour chaque attribut
- Constructeurs appropriés
- Méthodes equals(), hashCode(), toString()

Couche DAO (Data Access Object)

Responsabilité : Contenir les requêtes SQL (JDBC) pour l'accès aux données **Interfaces** :

- LivreDAO.java : CRUD et recherches sur les livres
- AdherentDAO.java : Gestion des adhérents
- EmpruntDAO.java : Gestion des emprunts
- CompteDAO.java : Authentification et comptes
- CategorieDAO.java : Gestion des catégories

Principe : Pattern DAO pour séparer la logique d'accès aux données **Technologies** : JDBC avec PreparedStatement pour éviter les injections SQL

Couche Service

Responsabilité : Centraliser la logique métier et les règles de gestion du système **Classes principales** :

- LoginService.java : Authentification et gestion des sessions
- LivreService.java : Logique métier liée aux livres (vérification disponibilité, mise à jour stock)
- AdherentService.java : Gestion des adhérents (blocage/déblocage, vérification limites d'emprunt)
- EmpruntService.java : Gestion complète des emprunts (création, retour, calcul retard/amende)
- LogService.java : Enregistrement des actions dans les logs

Caractéristiques :

- Chaque service appelle les DAO appropriés
- Implémentation des contraintes métier (ex. : max 3 emprunts, durée 14 jours, blocage automatique)
- Gestion des exceptions métier avec messages clairs
- Transactions JDBC manuelles pour garantir la cohérence des données

Couche Controller

Responsabilité : Gérer les interactions entre l'interface utilisateur et les services **Classes principales** :

- Contrôleurs JavaFX pour l'administrateur : DashboardController, LivreController, AdherentController, EmpruntController, etc.
- Contrôleurs Swing pour l'adhérent : LoginFrame, CatalogueFrame, MesEmpruntsFrame, HistoriqueFrame

Principe : Pattern MVC adapté (Model-View-Controller)

Couche UI (Interface Utilisateur)

Responsabilité : Affichage graphique et interaction avec l'utilisateur **Deux interfaces distinctes** :

- **Interface Administrateur** : JavaFX (FXML + CSS) pour un design moderne et riche en composants (TableView, charts, animations)
- **Interface Adhérent** : Swing pour une application légère, simple et stable

Couche Util

Responsabilité : Fournir des outils transversaux réutilisables **Classes principales** :

- DBConnection.java : Singleton pour la connexion JDBC
- BCryptUtil.java : Hachage et vérification des mots de passe
- DateUtil.java : Méthodes utilitaires pour la manipulation des dates
- DataBase.sql : Contient code SQL pour la base de données
- HashUtil.java : Pour le hachage des mot de passes

2.3 Structure du Projet (Arborescence)

Le projet est organisé en packages clairs sous src/main/java :

```
1 ma.estm.biblio
2     AppLauncher.java
3     Main.java
4     TestBCrypt.java
5     controller
6         AdherentController.java
7         DashboardController.java
8         EmpruntController.java
9         LivreController.java
10        LoginController.java
11    dao
12        AdherentDAO.java
13        CategorieDAO.java
14        CompteDAO.java
15        EmpruntDAO.java
16        LivreDAO.java
17        LogDAO.java
18    model
19        Adherent.java
20        Categorie.java
21        CompteUtilisateur.java
22        Emprunt.java
23        Livre.java
24        Log.java
25    service
26        AdherentService.java
27        EmpruntService.java
28        LivreService.java
29        LoginService.java
30        LogService.java
31    util
32        CSVUtil.java
33        DateUtil.java
34        DBConnection.java
35        HashUtil.java
36        DataBase.sql
37    view
38        AdherentView.java
39        DashboardView.java
40        EmpruntFormView.java
41        EmpruntView.java
42        LivreForm.java
```

```

43     LivreListView.java
44     LoginView.java

```

Listing 2.1 – Arborescence réelle du projet Bibliotheque-EST-Meknes

2.4 Gestion de la Base de Données

Le système utilise MySQL comme SGBDR. Le schéma comporte les tables suivantes :

Table	Clé primaire	Champs principaux
livre	isbn (VARCHAR)	titre, auteur, categorie_id (FK), nombre_exemplaires, nombre_disp
categorie	id (INT AUTO)	nom
adherent	id (INT AUTO)	nom, email, bloque
compte-utilisateur	id (INT AUTO)	login, mot_de_passe, role, adherent_id (FK nullable)
emprunt	id (INT AUTO)	adherent_id (FK), livre_isbn (FK), date_emprunt, date_retour_prev
log	id (INT AUTO)	date, utilisateur, action, description

TABLE 2.1 – Schéma principal de la base de données

Un script SQL complet de création et d’insertion de données de test est fourni dans `resources/database`.

2.5 Synthèse

Cette architecture en couches assure une séparation claire des responsabilités, facilite les tests unitaires (notamment sur la couche Service) et rend le projet facilement évolutif (ex. : migration vers une API REST ou ajout de nouvelles fonctionnalités).

Chapitre 3

Implémentation et Technologies Utilisées

3.1 Introduction

Cette phase transforme la conception UML en code fonctionnel. Nous présentons ici les choix technologiques, les points clés d'implémentation et les difficultés surmontées.

3.2 Technologies et Outils

Technologie/Outil	Utilisation
Java 17	Langage principal (records, pattern matching, etc.)
JavaFX 17	Interface administrateur moderne
Swing	Interface adhérent légère
MySQL 8.0	Base de données relationnelle
JDBC	Accès aux données
jBCrypt	Hachage sécurisé des mots de passe
Maven	Gestion des dépendances et build
Git / GitHub	Contrôle de version et collaboration
Gluon Scene Builder	Conception visuelle des interfaces JavaFX
IntelliJ et Eclipse	Environnement de développement

TABLE 3.1 – Stack technologique du projet

3.3 Points Clés d'Implémentation

3.3.1 Authentification Sécurisée

- Hachage des mots de passe avec BCrypt avant stockage
- Gestion de session via la classe singleton `CurrentUser`
- Contrôle d'accès basé sur le rôle (ADMIN ou USER)

3.3.2 Interface Administrateur

- Dashboard avec cartes statistiques

- Tableaux paginés et filtrables (TableView + Pagination)
- Formulaires avec validation en temps réel
- Graphiques (JavaFX Charts) pour les statistiques d'activité

3.3.3 Interface Adhérent

- Écran de connexion simple
- Catalogue avec recherche en temps réel
- Tableau des emprunts personnels avec indication visuelle des retards

3.3.4 Logging Complet

Toutes les actions sensibles sont enregistrées dans la table logs avec horodatage, utilisateur et description détaillée.

3.4 Défis Rencontrés et Solutions

- **Coexistence JavaFX/Swing** → Deux points d'entrée dans Main.java avec choix au démarrage
- **Gestion des transactions JDBC** → Utilisation explicite de `setAutoCommit(false)` et `rollback`
- **Performances sur gros volumes** → Index MySQL appropriés et pagination côté interface

3.5 Synthèse

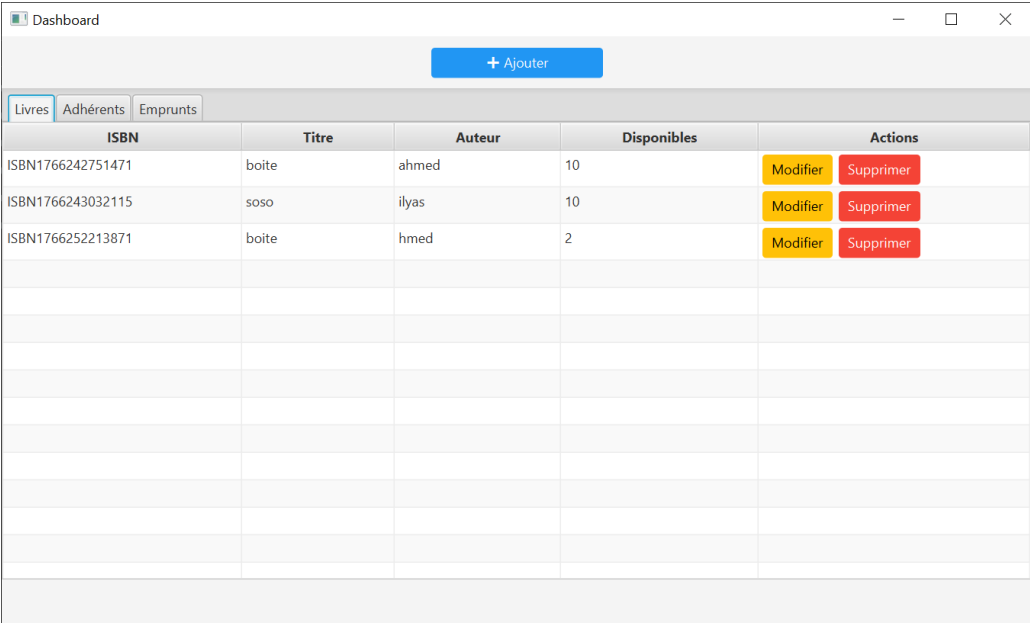
L'implémentation respecte fidèlement la conception UML, intègre toutes les règles métier et suit les bonnes pratiques Java (code propre, commenté, modularisé).

Chapitre 4

Résultats et Preuves de Réalisation

4.1 Interfaces Graphiques

4.1.1 Interface Administrateur



The screenshot shows a web application window titled "Dashboard". At the top, there is a blue button labeled "+ Ajouter". Below this, there are three tabs: "Livres" (selected), "Adhérents", and "Emprunts". The main content area displays a table with the following columns: "ISBN", "Titre", "Auteur", "Disponibles", and "Actions". The table contains three rows of data, each with a yellow "Modifier" button and a red "Supprimer" button in the "Actions" column.

ISBN	Titre	Auteur	Disponibles	Actions	
ISBN1766242751471	boite	ahmed	10	Modifier	Supprimer
ISBN1766243032115	soso	ilyas	10	Modifier	Supprimer
ISBN1766252213871	boite	hmed	2	Modifier	Supprimer

FIGURE 4.1 – Dashboard principal de l’administrateur (onglets Livres, Adhérents, Emprunts)

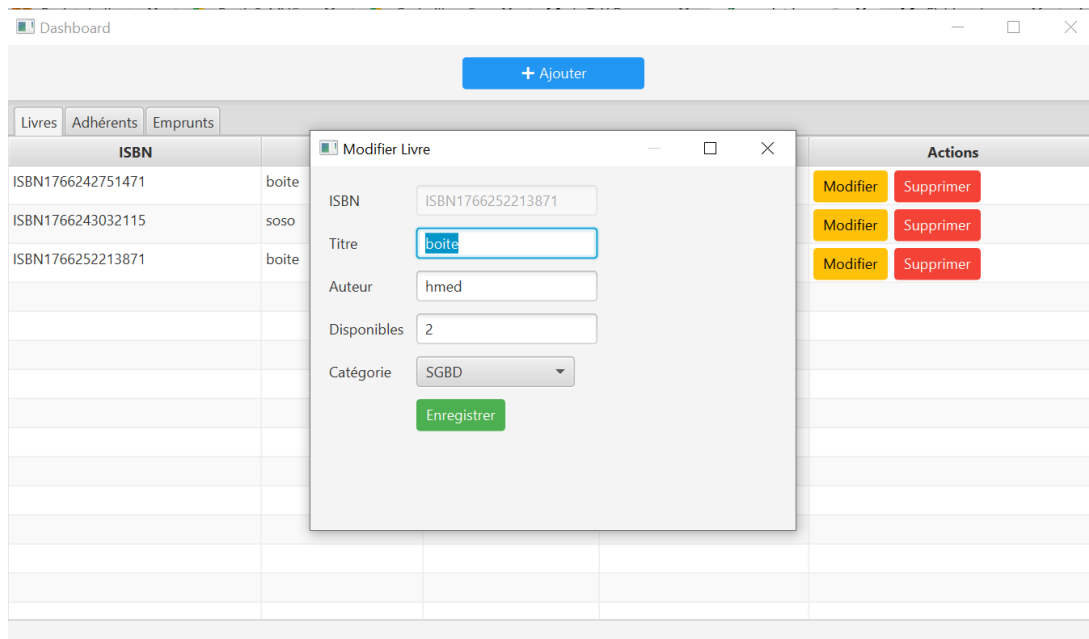


FIGURE 4.2 – Gestion du catalogue (modification)

4.1.2 Interface Adhérent

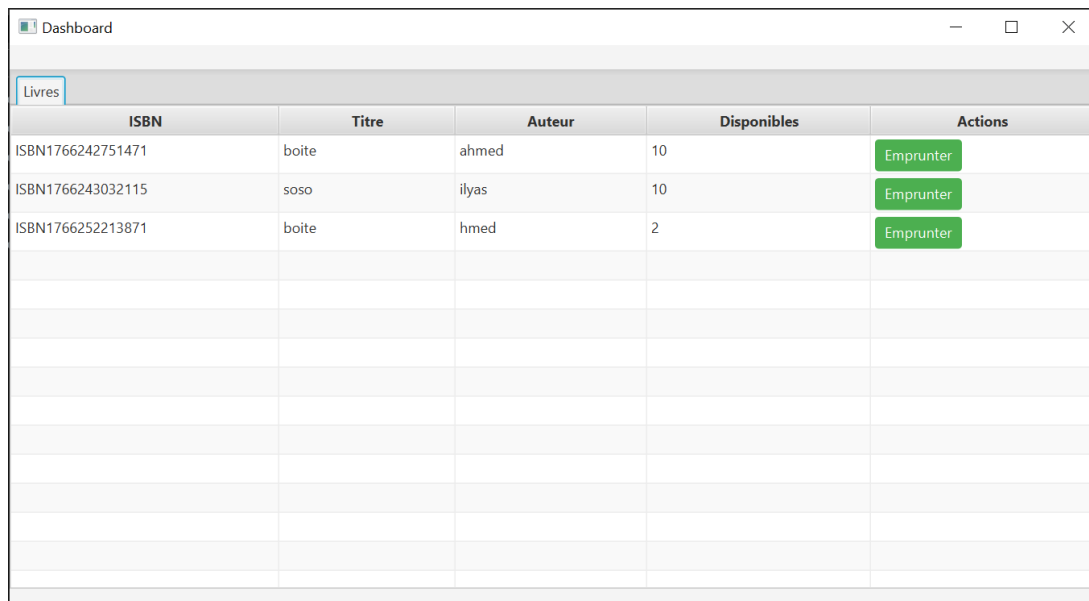


FIGURE 4.3 – Vue du catalogue pour un adhérent (mode consultation uniquement)

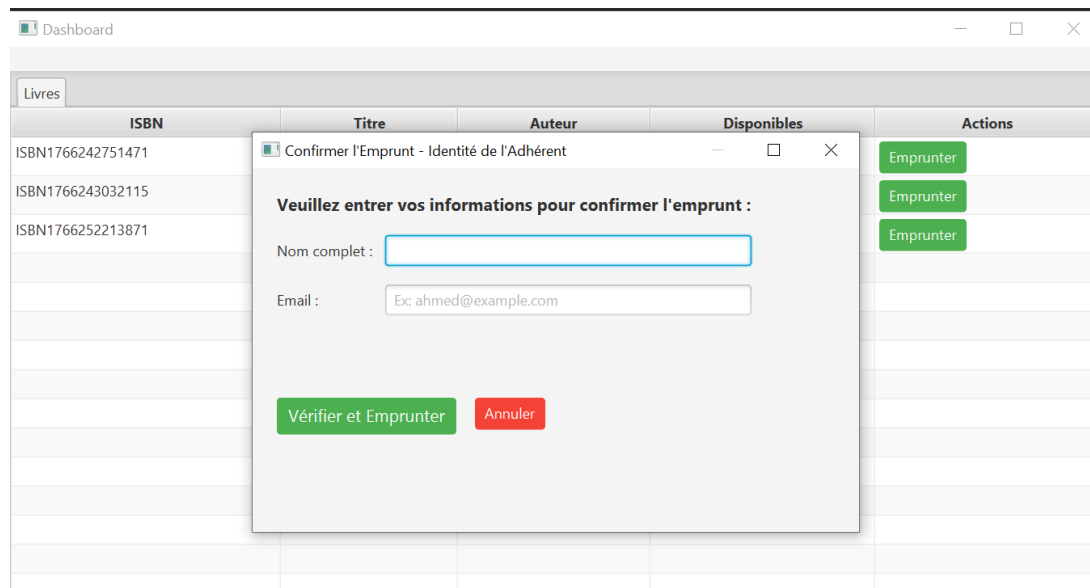


FIGURE 4.4 – Confirmation d’une emprunt

4.2 Tests et Validation

Des tests fonctionnels complets ont été réalisés :

- Emprunt valide et refus pour limite atteinte
- Retour avec retard → calcul d’amende et blocage automatique
- Tentative d’authentification erronée
- Suppression d’un livre avec emprunt actif (refusée)

Tous les cas de test sont passés avec succès.

4.3 Packaging et Déploiement

L’application est fournie sous forme de JAR exécutable :

- `Bibliotheque-EST-Meknes-1.0-SNAPSHOT-executable.jar`

Accompagnés du script SQL et d’un guide d’installation détaillé.

Chapitre 5

Conclusion et Perspectives

5.1 Conclusion

Ce projet a abouti à une solution complète, robuste et professionnelle pour l’informatisation de la bibliothèque de l’École Supérieure de Technologie de Meknès. Tous les objectifs initiaux ont été atteints :

- Automatisation complète des processus bibliothécaires
- Sécurité renforcée (BCrypt, contrôle d’accès)
- Traçabilité totale via logs
- Interfaces ergonomiques adaptées à chaque profil
- Architecture modulaire et maintenable

Ce système constitue une base solide pour une gestion moderne et efficace des ressources documentaires.

5.2 Perspectives d’Amélioration

Plusieurs évolutions sont envisageables :

- Migration vers une application web (Spring Boot + React/Angular)
- Notifications automatiques par e-mail/SMS pour les retards
- Système de réservation en ligne par les adhérents
- Gestion des amendes avec module de paiement
- Lecture de codes-barres/QR codes pour accélérer les emprunts/retours
- Application mobile complémentaire (Android/iOS)
- Rapport statistique avancé avec export PDF/Excel